LangChain provides higher-level abstractions for agents, tool management, and reasoning loops. With its MCP adapters, we can plug MCP tools into LangChain's agent ecosystem with minimal effort. The result is a more flexible orchestration layer where the AI can decide when to invoke tools and how to combine their results.

Let's see an example of how we can integrate LangChain with the Vercel AI SDK and provide a web application that allows users to ask for jokes about Chuck Norris. The jokes will be sourced from the Chuck Norris API (https://api.chucknorris.io/) into our application.

Project code: langchain-mcp

The code for this example project can be found in the ch12/langchain-mcp folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch12/langchain-mcp
```

You'll need the Gemini API key or OpenAI key for this to work as the application uses services from different vendors. These are configured as environment variables:

```
// .env
GOOGLE_API_KEY=<GEMINI_API_SECRET_KEY>
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
```

Instructions on how to set up and create secret keys for Google AI and OpenAI are provided in the appendix.

12.5.1 Architecture overview

Figure 12.7 shows the overall system design for the LangChain.js with MCP integration.

The architecture layers are like the Vercel AI SDK integration but with some key differences:

- *LangChain agent*—Instead of directly streaming responses from the SDK, LangChain creates an agent that can reason about when to use tools (`createReactAgent`).
- *MCP integration via adapter*—LangChain uses the `MultiServerMCPClient` to fetch MCP tools and expose them as LangChain tools.

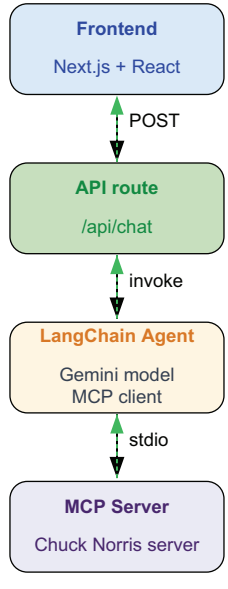


Figure 12.7 High-level architecture of a Next.js chat application using LangChain.js and MCP. The frontend interacts with a Next.js API route. This route constructs a LangChain agent that uses an MCP client. The MCP client connects to the Chuck Norris MCP server over `stdio` and invokes tools on behalf of the model.